

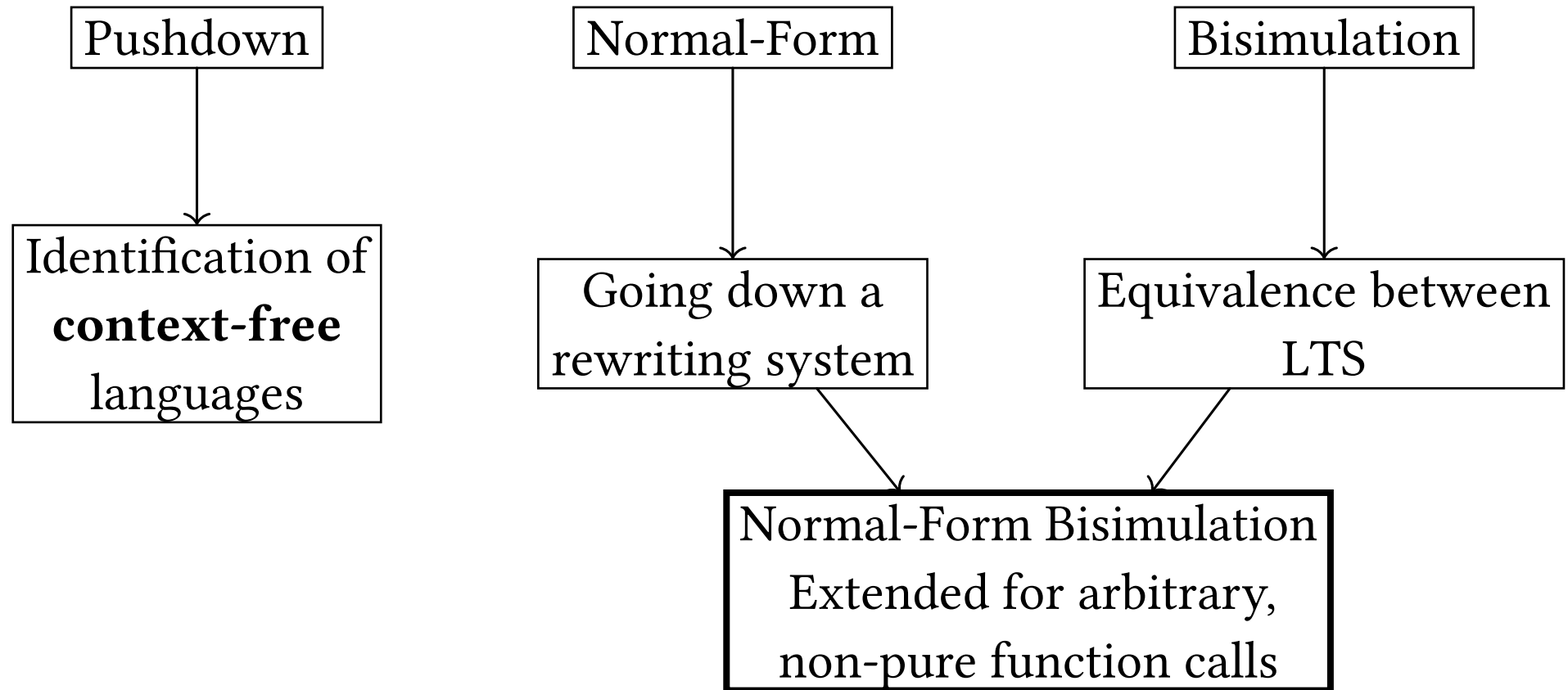
Pushdown Normal Form Bisimulation 2

Electric Boogaloo

A Nominal, Context Free Approach to program equivalence

By Fares Mestre, based on the paper by Vaselios Koutavas, Yuyang Lin, and Nikos Tzevelekos [1]

Previously...



So that

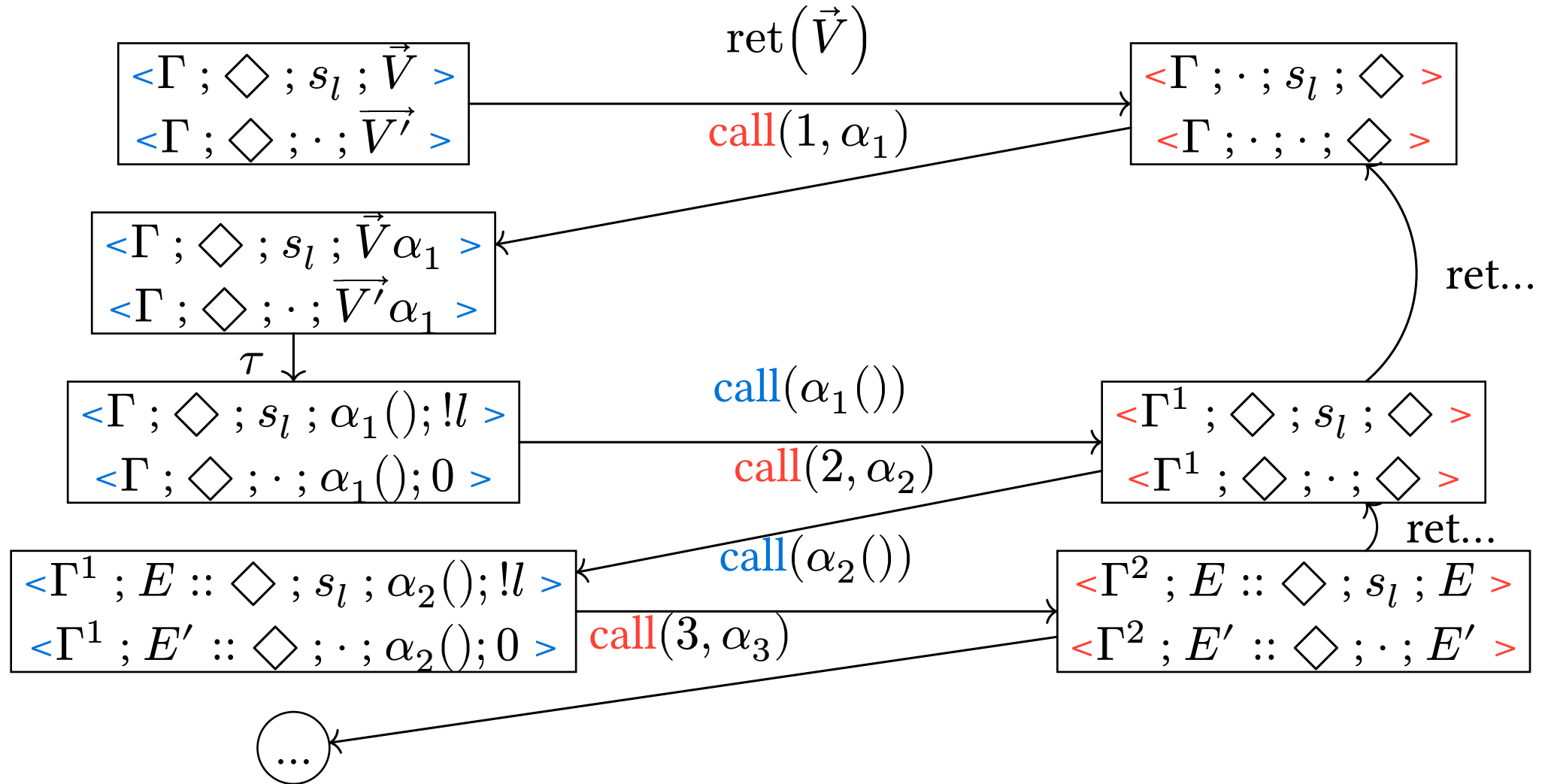
Bisimilarity $\Leftrightarrow$ Contextual equivalence

Previously...



- Verifies program equivalence
- Finds inequivalences

Previously...



The problem...



The solution



How do we blow up
pancakes with mind ?

How do we get rid of the stack ?

Can we even do that ?

$$\text{PROP RET} : \langle \Gamma ; \xi, K ; s ; v \rangle \xrightarrow{\text{ret } (D)} \langle \Gamma, \Gamma' ; K ; s ; \xi \rangle$$

$$\text{OPP CALL} : \langle \Gamma ; K ; s ; \xi \rangle \xrightarrow{\text{call } (i, D[\vec{\alpha}])} \langle \Gamma ; \xi, K ; s ; e \rangle$$

- Only **proponent** returns and **opponent** calls manipulate the stack
- The **opponent** issues a challenge ξ_1 , and the **proponent** answers with ξ_2
 - Neither factor in the previous stack

Yes we can!

How do we get rid of the stack ?

Stackless configurations

$$\text{PROP_RET} : \langle \Gamma ; s ; v \rangle \xrightarrow{\text{ret } (D)} \langle \Gamma, \Gamma' ; s ; \chi \rangle$$

$$\text{OPP_CALL} : \langle \Gamma ; s ; \xi \rangle \xrightarrow{\text{call } (i, D[\vec{\alpha}])} \langle \Gamma ; s ; e \rangle$$

- Everything else, including movement properties, is the same, only with no stack
- χ works as a placeholder (this will make more sense later)

Did we actually get rid of the stack ?

- We still have to account for some sort of stack discipline if we want to be somewhat correct
- If it's not going to be in configurations, then we have to move both stacks into the bisimulation game

What do we actually want from removing the stack ?

$$\begin{array}{l} \langle \Gamma ; | ; s ; \xi_1 \rangle \\ \langle \Gamma ; || ; s ; \xi_2 \rangle \\ \langle \Gamma ; ||| ; s ; \xi_3 \rangle \\ \langle \Gamma ; |||| ; s ; \xi_4 \rangle \\ \langle \Gamma ; ||||| ; s ; \xi_5 \rangle \\ \dots \end{array} \xRightarrow{\text{Saturation algorithms [2]}} \langle \Gamma ; s ; \chi \rangle$$

By finitizing the LTS, we can automate verification!

Remember our prior solution ?



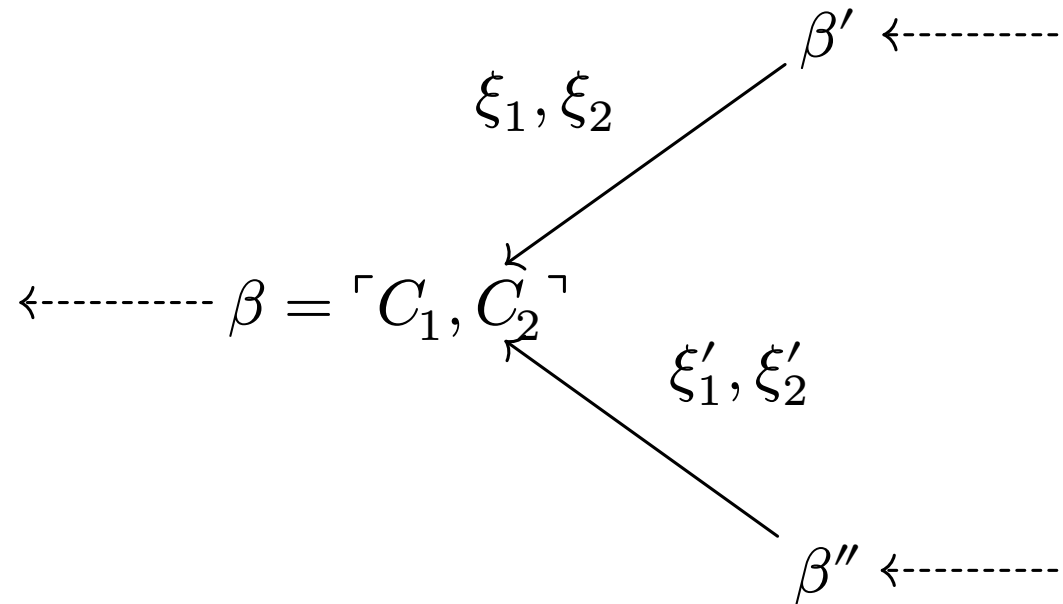
It had consequences...



But that's exactly what we want!



Continuation graphs



- Vertices β can be seen as a pair of **proponent** configurations reached after an **opponent** call (or be empty $\diamond$)
- Edges $\beta' \xrightarrow{\xi_1, \xi_2} \beta$ can be seen as the summary of a challenge

$$\ulcorner C_1, C_2 \urcorner \xrightarrow{\text{PropRet}} \ulcorner (\Gamma ; s ; \xi_1), (\Gamma ; s ; \xi_2) \urcorner \xrightarrow{\text{OppCall}} \ulcorner C'_1, C'_2 \urcorner$$

Changes to the Bisimulation game

- We go from configuration pairs (C_1, C_2) to tuples $\langle C_1, C_2, \Sigma, \beta \rangle$
 - C_1 and C_2 are stackless configurations
 - Σ is a continuation graph
 - β is the current entry point in the continuation graph

- Push :

$$\langle C_1, C_2, \Sigma, \beta \rangle \xrightarrow{\text{OpCall}} \langle C'_1, C'_2, \Sigma \left[\beta' \xrightarrow{\xi_1, \xi_2} \beta \right], \beta' \rangle$$

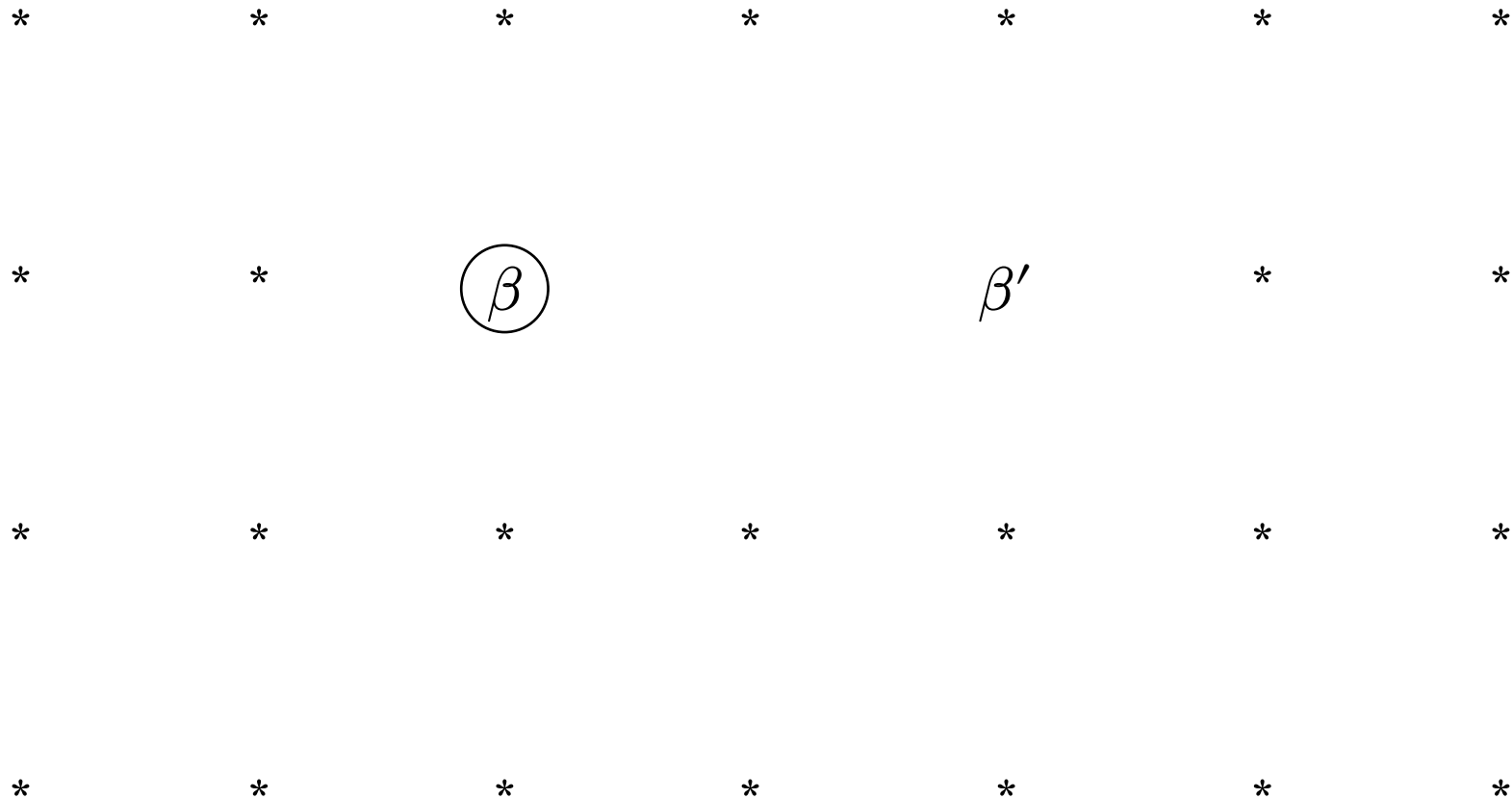
- Pop :

$$\langle \hat{C}_1, \hat{C}_2, \Sigma, \beta' \rangle \xrightarrow{\text{PropRet}} \langle C_1[\xi_1/\chi], C_2[\xi_2/\chi], \Sigma, \beta \rangle$$

if β is accessible from β' using ξ_1, ξ_2

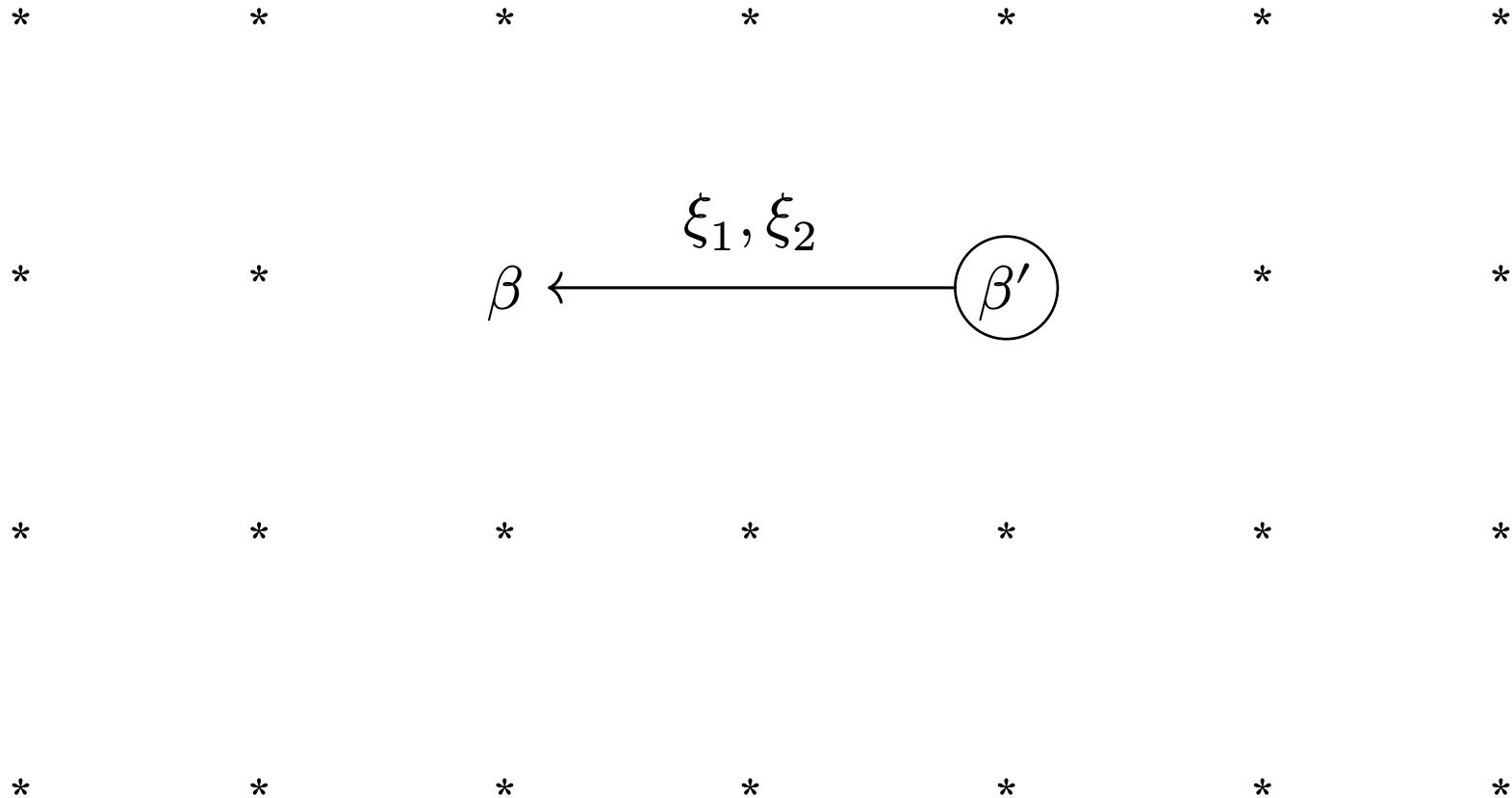
Changes to the Bisimulation game

Starting...



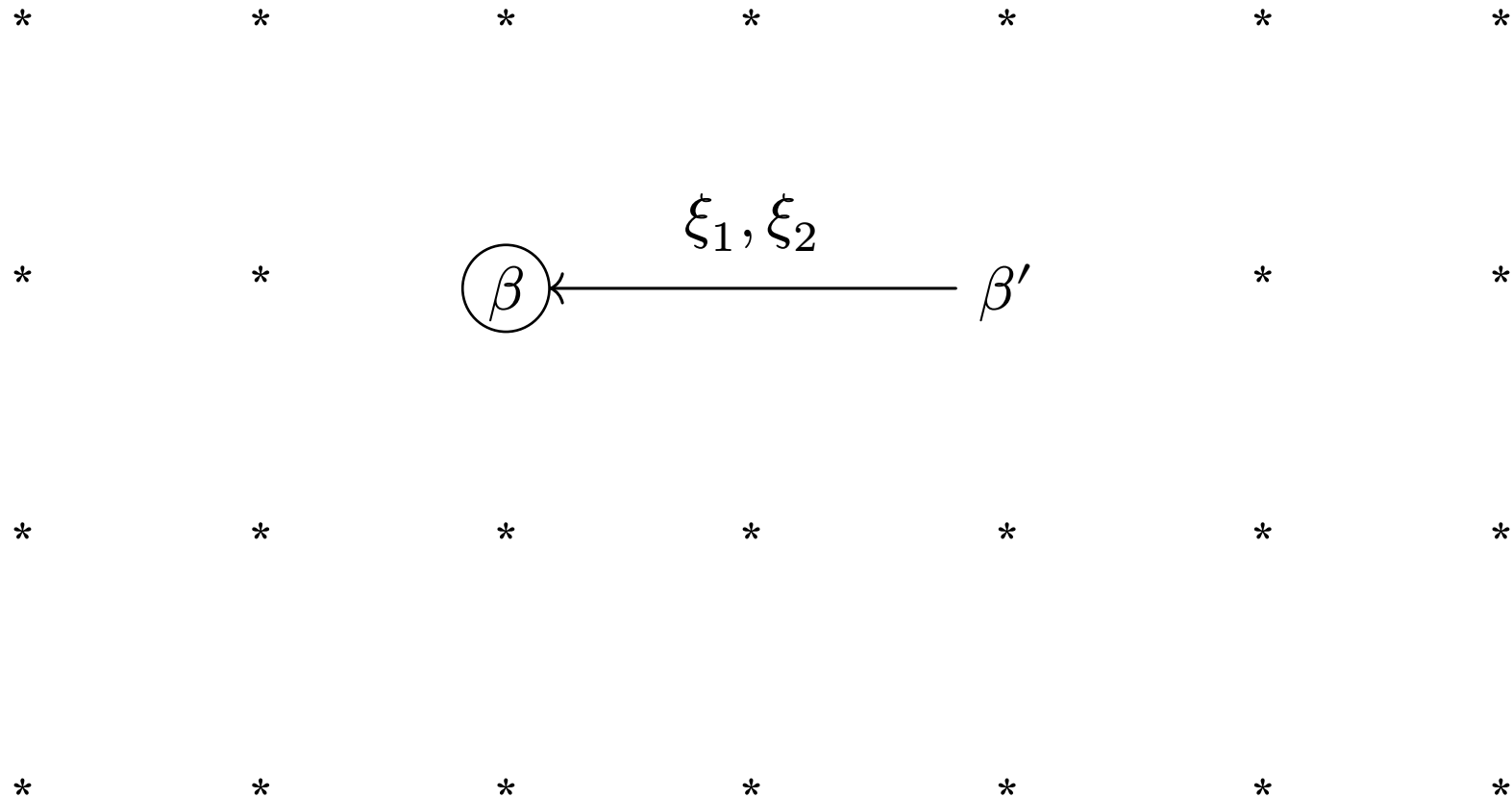
Changes to the Bisimulation game

OPCALL



Changes to the Bisimulation game

PROP_{RET}



Properties on continuation graphs Σ

- $\Sigma \neq \emptyset$
- Σ is **orbit-finite**
- Reachability : $\forall \beta \in \text{dom}(\Sigma), \exists K_1, K_2$: Evaluation stacks such that

$$\beta \xrightarrow{K_1, K_2} {}^*_\Sigma \Diamond$$

- In particular, $\Diamond \xrightarrow{\cdot, \cdot} {}^*_\Sigma \Diamond$ and

$$\beta' \xrightarrow{\xi_1 :: K_1, \xi_2 :: K_2} \Diamond \Rightarrow \beta' \xrightarrow{\xi_1, \xi_2} {}_\Sigma \beta \xrightarrow{K_1, K_2} {}^*_\Sigma \Diamond$$

Properties on continuation graphs Σ

- Top : $\forall \beta' \xrightarrow{\xi_1, \xi_2}_{\Sigma} \beta$ and $j \in \{1, 2\}$
 - $\diamond \in \text{dom}(\Sigma)$
 - $\beta' = \diamond \Rightarrow \beta = \diamond$
 - $\beta = \diamond \Leftrightarrow \xi_j = \diamond$
- Divergence :
 - $\beta'.j = \perp \Leftrightarrow \xi_j = \perp$
 - $\beta.j = \perp \Rightarrow \xi_j = \perp$
- Nominal closure :
 - Σ contains all **permutations** of an edge $\beta' \xrightarrow{\xi_1, \xi_2} \beta \in \Sigma$

These properties guarantee Σ only produces valid, compatible stacks

Corollaries

- Σ will always contain the loop

$$\Sigma_{\diamond} = \begin{array}{c} \diamond, \diamond \\ \curvearrowright \\ \diamond \end{array}$$

Which can be seen as the **empty stack**

- The Nominal Closure property allows us to **saturate** the continuation graph in a finite amount of steps
 - $\rightarrow$ Allows for **nominal** treatment of the graph
 - $\rightarrow$ Lets us work on a pushdown nominal automata
 - $\rightarrow$ Gives us decideability for context-free expressions [3]

PDFN Bisimulation

- Compatible (bi)simulation tuples : $\langle C_1, C_2, \Sigma, \beta \rangle$ is compatible when
 - C_1 and C_2 are compatible (same polarity, same $\text{dom}(\Gamma)$, and $C_1.\xi = \diamond \Leftrightarrow C_2.\xi = \diamond$, or at least one diverges)
 - Restricting Σ to its reachable subgraph from β ($\Sigma@ \beta$) leaves it unchanged
 - For $j \in \{1, 2\}$
 - $\beta.j = \perp \Rightarrow C_j = (\perp)$
 - $\beta = \diamond \Rightarrow C_j.\xi = \diamond \vee C_j.e$ has no abstract name
 - $C_j.\xi = \diamond \Rightarrow \beta = \diamond$

PDNF Bisimulation

A relation between compatible tuples $\langle C_1, C_2, \Sigma, \beta \rangle$, noted $C_1 R_{\Sigma, \beta} C_2$, is a weak simulation when for all $C_1 R_{\Sigma, \beta} C_2$,

1. $C_1 = (\Gamma ; s ; \diamond) \implies C_2' = (\Gamma ; s ; \diamond)$
2. $C_1 \xrightarrow{\text{ret}(D[\vec{\alpha}])} C_1'$ with $\vec{\alpha} \# C_2 \implies C_2 \xrightarrow{\text{ret}(D[\vec{\alpha}])} C_2'$ such that $C_1' R_{\Sigma, \beta} C_2'$
3. For $\eta \in \{\tau, \text{call}(\alpha, D)\}$, $C_1 \xrightarrow{\eta} C_1' \implies C_2 \xrightarrow{\tau^*; \eta; \tau^*} C_2' \wedge C_1' R_{\Sigma, \beta} C_2'$
4. $C_1 \xrightarrow{\text{call}(i, D[\vec{\alpha}])} C_1'$ with $\vec{\alpha} \# C_2, \beta \implies C_2 \xrightarrow{\text{call}(i, D[\vec{\alpha}])} C_2'$ such that $C_1' R_{\Sigma', \beta'} C_2'$
 - $\beta' = \ulcorner C_1', C_2' \urcorner$ and $\Sigma' = \Sigma \left[\beta' \xrightarrow{C_1 \cdot \xi, C_2 \cdot \xi} \beta \right]$
5. $C_1 \xrightarrow{\text{ret}(D)} C_1' \wedge \beta \xrightarrow{\xi_1, \xi_2} \beta' \implies C_2 \xrightarrow{\tau^*; \text{ret}(D) \tau^*} C_2' \wedge C_1'[\xi_1/\chi] R_{\Sigma, \beta} C_2'[\xi_2/\chi]$

PDNF Bisimulation

- If $R = R^\circ$, with $\langle C_1, C_2, \Sigma, \beta \rangle^\circ = \langle C_2, C_1, \Sigma^\circ, \beta^\circ \rangle$, R is a bisimulation
- Bisimilarity ($\approx$) is the largest bisimulation
- Expressions $\vdash e_1, e_2 : T$ with no abstract name are PDNF bisimilar
 $(e_1 \approx_{\text{PD}} e_2)$ when $\llbracket \cdot ; \cdot ; e_1 \rrbracket \approx_{\Sigma_{\diamond, \diamond}} \llbracket \cdot ; \cdot ; e_2 \rrbracket$

Finally, we defined what the paper is named after



Surely there are no more definitions

But our work is not done yet!

- We wanted two things
 - An automatic way to verify program equivalence
 - Equivalence between bisimilarity and contextual equivalence
- For that, we need to prove 3 properties on PDNF-Bisimulation
 - Decidability
 - Soundness
 - Completeness

PDNF Bisimulation decidability

Decidability on *bounded* PDNF Bisimulation

- What is bounded ? The size of an expression $\|e\|$
- A tuple $\langle C_1, C_2, \Sigma, \beta \rangle$ is k -bounded if all of its continuation graph's edges are
- A candidate bisimulation R is k -bounded if all its tuples are

Lemma : If R is k -bounded and equivariant then it is orbit-finite



Permutations and nominal treatment

- We consider finitary permutations over store locations, abstract names, and environment indices
- An object X is considered **equivariant** if $\forall \pi \in \text{Perm}, \pi \cdot X = X$
- A set of objects is called **nominal** if it is closed under permutation
- The **orbit** of an object is $\text{orb}(X) = \{\pi \cdot X \mid \pi \in \text{Perm}\}$
- An object is **orbit-finite** if its orbit is finite (shocker)

Decidability on *bounded* PDNF bisimulation

Proof scheme

- The set of k -bounded configurations is orbit-finite by the definition of continuation graphs
- Orbit-finiteness in nominal sets is a property closed [4] under all operations used to define R

Now we need to prove the general case

Decidability of PDNF bisimulation

Theorem : For k -bounded, context-free expressions e_1, e_2 ,

$$e_1 \approx_{\text{PD}} e_2 \iff \langle \cdot ; \cdot ; e_1 \rangle R_{\Sigma_{\diamond, \diamond}} \langle \cdot ; \cdot ; e_2 \rangle$$

Proof scheme :

- $\implies$:
 - e_1, e_2 are k -bounded and context-free $\Rightarrow$ Bisimulation only reaches k -bounded tuples
 - Restrict $\approx_{\text{PD}}$ to k -bounded elements and use it as a candidate
- $\impliedby$:
 - Orbit-finiteness allows us to exhaustively build R
 - Pick a bound k
 - If a candidate fits all conditions but k -boundedness, restart with $k + 1$

PDNF Bisimulation soundness

More precisely, with regards to NF-Bisimulation

Proof scheme

- We define $(\widetilde{C}, K)$ the stacked extension by K of a stackless configuration C
- We then prove that if R is a simulation, then so is its stacked extension $\tilde{R}$

$$\tilde{R} = \left\{ (\widetilde{C_1}, K_1), (\widetilde{C_2}, K_2) \mid \exists \Sigma, \beta \text{ st } C_1 R_{\Sigma, \beta} C_2 \wedge \beta \xrightarrow{K_1, K_2}^*_{\Sigma} \diamond \right\}$$

- Once this is proven, it easily carries over to bisimulation as well

PDNF Bisimulation completeness

Saturated Simulation LTS

- In a nutshell, it lets us put ε -transitions into the LTS
- Also guarantees reflexivity and transitivity
- It is defined inductively (Section)

Proof scheme

- We prove by induction that similarity on stacked extensions is closed under transitions in the saturated LTS
- Then, we prove equivalence between stackless and stack-extended simulations

Corollary

- Contextual equivalence is decidable for context-free expressions

Hobbit-PDNF

Changelog

- Now implements stackless LTS instead of stacked
- Implements a bounded, symbolic execution of the PDNF Bisimulation game
- Up-To Reentry technique has been redefined ; finitizing reentrances of pure function calls is no longer needed
- Bounds are akin to a timeout condition
 - Additionnally, it's not just the number of calls that is bounded
 - Proponent calls
 - Opponent calls
 - Opponent returns
 - This lets us avoid cycles in the continuation graph

Results on equivalence verification

	HOBBIT : 129 $\approx$ + 78 $\not\approx$			HOBBIT-PDNF : 11 $\approx$	
	PDNF	HOBBIT	H + Reentry	PDNF	HOBBIT
Eq. proven	72	62	67	11	0
Ineq. proven	77	78	78	N/A	N/A

- Testing HOBBIT-PDNF on the original HOBBIT test suite reveals it can automatically prove more equivalences
 - Even when assisted with reentry notation
 - With minimal impact on performance
- HOBBIT-PDNF comes with its own test suite, mostly comprised of tests inspired by the event-handler pattern
 - Those were inspired by Android, Javascript, Java Swing, jQuery, etc...
 - It verified all equivalences, whereas the original couldn't

Results on equivalence verification

	HOBBIT : $129 \approx + 78 \not\approx$			HOBBIT-PDNF : $11 \approx$	
	PDNF	HOBBIT	H + Reentry	PDNF	HOBBIT
Eq. proven	72	62	67	11	0
Ineq. proven	77	78	78	N/A	N/A

- HOBBIT supports invariant notation
 - If helped, it can prove some equivalences HOBBIT-PDNF can't
- HOBBIT-PDNF may also explore more spurious paths than the original
 - Might explain the one inequivalence it couldn't verify over the original

Bibliography

- [1] V. Koutavas, Y.-Y. Lin, and N. Tzevelekos, “Pushdown Normal-Form Bisimulation: A Nominal Context-Free Approach to Program Equivalence,” in *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science*, in LICS '24. Tallinn, Estonia: Association for Computing Machinery, 2024. doi: 10.1145/3661814.3662103.
- [2] A. Bouajjani, J. Esparza, and O. Maler, “Reachability analysis of pushdown automata: Application to model-checking,” in *CONCUR '97: Concurrency Theory*, A. Mazurkiewicz and J. Winkowski, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1997, pp. 135–150.
- [3] A. Murawski, S. Ramsay, and N. Tzevelekos, “Reachability in pushdown register automata,” *Journal of Computer and System Sciences*, vol. 87, pp. 58–83, 2017, doi: <https://doi.org/10.1016/j.jcss.2017.02.008>.

- [4] M. Bojańczyk, B. Klin, and S. Lasota, “Automata theory in nominal sets,” *Logical Methods in Computer Science*, Aug. 2014, doi: 10.2168/LMCS-10(3:4)2014.

Saturated Simulation LTS

TAU

$$\frac{C_i \xrightarrow{\tau} C'_i \quad C_{3-i} = C'_{3-i}}{(C_1, C_2) \xrightarrow[\text{SAT}]{\varepsilon} (C'_1, C'_2)}$$

PROPCALL

$$\frac{C_1 \xrightarrow{\text{call}(\alpha, D)} C'_1 \quad C_2 \xrightarrow{\text{call}(\alpha, D)} C'_2}{(C_1, C_2) \xrightarrow[\text{SAT}]{\varepsilon} (C'_1, C'_2)}$$

OPCALL

$$\frac{C_1 \xrightarrow{\text{call}(i, D[\vec{\alpha}])} C'_j \quad C_2 \xrightarrow{\text{call}(i, D[\vec{\alpha}])} C'_2}{(C_1, C_2) \xrightarrow[\text{SAT}]{C_1 \cdot \xi, C_2 \cdot \xi} (C'_1, C'_2)}$$

OPRET

$$\frac{C_1 \xrightarrow{\text{ret}(D[\vec{\alpha}])} C'_1 \quad C_2 \xrightarrow{\text{ret}(D[\vec{\alpha}])} C'_2}{(C_1, C_2) \xrightarrow[\text{SAT}]{\varepsilon} (C'_1, C'_2)}$$

Saturated Simulation LTS

PROP_{RET}

$$\frac{(C_1, C_2) \xrightarrow[\text{SAT}]{\xi_1, \xi_2} (C'_1, C'_2) \xrightarrow[\text{SAT}]{\varepsilon} (C''_1, C''_2) \quad C''_1 \xrightarrow{\text{ret}(D)} C'''_1 \quad C''_2 \xrightarrow{\text{ret}(D)} C'''_2}{(C_1, C_2) \xrightarrow[\text{SAT}]{\varepsilon} (C'''_1[\xi_1/\chi], C'''_2[\xi_2/\chi])}$$

TRANS

REFL

$$\frac{}{(C_1, C_2) \xrightarrow[\text{SAT}]{\varepsilon} (C_1, C_2)} \quad \frac{(C_1, C_2) \xrightarrow[\text{SAT}]{\varepsilon} (C'_1, C'_2) \xrightarrow[\text{SAT}]{\varepsilon} (C''_1, C''_2)}{(C_1, C_2) \xrightarrow[\text{SAT}]{\varepsilon} (C''_1, C''_2)}$$

Section